



DEEP LEARNING AND NATURAL LANGUAGE PROCESSING

LECTURE 9: CONTROLLING AND ADAPTING LARGE LANGUAGE MODELS

Dr. Adewale Akinfaderin

SEAS 8520-D6A · Summer 2026



Table of Contents

1 The Adaptation Problem

- ▶ The Adaptation Problem
- ▶ Generation Control: Decoding Strategies
- ▶ Advanced Prompting
- ▶ Supervised Fine-Tuning
- ▶ Parameter-Efficient Fine-Tuning and Quantization
- ▶ Case Study: Prompting vs. Fine-Tuning on a Laptop
- ▶ Alignment: Learning from Preferences

Where we left off. The transformer gave us contextual representations, and pretraining on next-token prediction gave us large language models: networks that assign a probability to every possible continuation of a text. Today we ask the practical question that dominates applied NLP: **how do you make a pretrained model do what you want?**

1. **The adaptation problem:** base models predict text; users need assistants.
2. **Generation control:** greedy, beam, temperature, top- k , nucleus sampling.
3. **Advanced prompting:** in-context learning, chain-of-thought, tree of thoughts.
4. **Supervised fine-tuning:** changing the weights, and what it costs.
5. **PEFT and quantization:** LoRA, QLoRA, and low-precision arithmetic.
6. **Alignment:** RLHF and direct preference optimization.

The Gap Between a Text Predictor and an Assistant

1 The Adaptation Problem

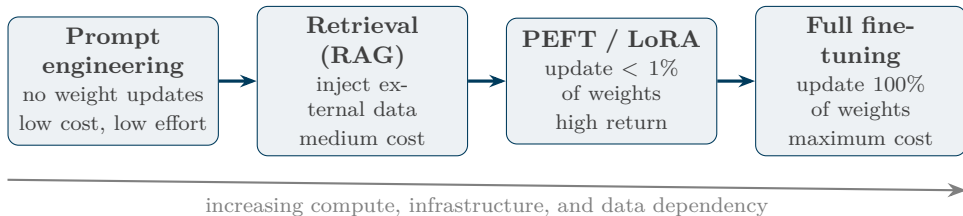
Pretraining produces a **text predictor**: given a prefix, it outputs a probability distribution over the next token. That objective says nothing about being useful.

- Prompt a base model with “*What is the capital of France?*” and a plausible continuation is another list of quiz questions, because quizzes are what that text pattern resembles in the training corpus.
- The model has enormous latent capability but no notion of **instructions**, **dialogue turns**, or **refusals**.
- Everything in today’s lecture is a technique for closing this gap, ordered from the cheapest intervention to the most expensive.

Today’s framing

Four levers, in increasing order of cost: control the **sampler**, control the **prompt**, change the **weights**, change the weights using **human preferences**.

Adapting a language model trades infrastructure complexity and compute against the ceiling on what you can change.



- Today covers the two ends: prompting on the left, weight updates on the right.
- Retrieval-augmented generation sits between the two and gets its own treatment later in this course.

The right intervention depends on *what* you are trying to change.

You want to change...	Reach for...	Why
Randomness, repetition, length	Decoding settings	No model call changes
Task framing, output format	Prompting	Fast iteration, no training
Knowledge of your documents	Retrieval (later)	Knowledge changes daily
Persistent style, domain skill	Fine-tuning	Behavior must survive any prompt
Which answers humans prefer	Preference tuning	No labeled “correct” output exists

- These compose: a production system typically fixes decoding, engineers the prompt, *and* serves a fine-tuned model.
- A common failure mode in practice is reaching for fine-tuning when a better prompt or a retrieval step would have sufficed at a fraction of the cost.



Table of Contents

2 Generation Control: Decoding Strategies

- ▶ The Adaptation Problem
- ▶ Generation Control: Decoding Strategies
- ▶ Advanced Prompting
- ▶ Supervised Fine-Tuning
- ▶ Parameter-Efficient Fine-Tuning and Quantization
- ▶ Case Study: Prompting vs. Fine-Tuning on a Laptop
- ▶ Alignment: Learning from Preferences

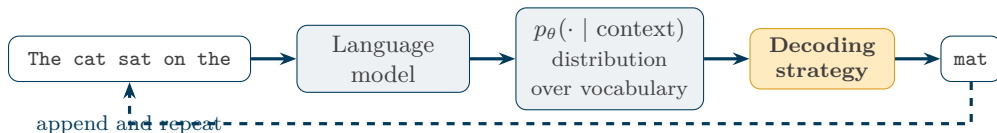
An autoregressive language model factorizes the probability of a sequence into next-token conditionals:

$$P(w_1, \dots, w_T) = \prod_{t=1}^T p_{\theta}(w_t \mid w_1, \dots, w_{t-1})$$

- A forward pass ends in a softmax over the vocabulary: 50,000 probabilities per step.
- The network never outputs text. **Some rule must pick one token** from that distribution, append it to the context, and repeat.
- That rule is the **decoding strategy**, and it is chosen by you, at inference time, with no retraining.

Why this matters

Two systems with *identical weights* can behave completely differently, one repetitive and robotic, the other fluent and creative, purely because of the decoding strategy.



- Feed a **conditioning prompt**, sample the next token, append it, and loop until a stop token or a length limit.
- Each pass through the loop is one full forward pass of the transformer.
- The loop structure is fixed; every method in this section only changes the **decoding strategy** box.

The simplest rule: always take the most probable token.

$$w_t = \arg \max_{w \in V} p_\theta(w \mid w_{<t})$$

- Deterministic: the same prompt always yields the same output.
- Equivalent to sampling from a distribution where the top token has probability 1 and every other token has probability 0.
- **The failure mode:** locally optimal choices compound into repetitive, predictable strings. Open-ended generation degenerates into loops (*“I don’t know. I don’t know. I don’t know.”*).
- Choosing the most likely token at each step does **not** produce the most likely *sequence*: the argmax of a product is not the product of argmaxes.

If greedy is short-sighted, search further ahead. **Beam search** maintains the B highest-scoring partial sequences (the *beam*) and extends each at every step, keeping the overall log-probability

$$\text{score}(w_{1:t}) = \sum_{i=1}^t \log p_{\theta}(w_i \mid w_{<i})$$

- With beam width B : expand each of the B hypotheses over the vocabulary, then keep the best B of the $B \times |V|$ candidates.
- Excellent for **closed-ended** tasks with a right answer: machine translation, summarization.
- For **open-ended** generation it makes degeneration *worse*: maximizing likelihood finds bland, repetitive text, because human language stays informative by continually surprising the reader (Holtzman et al., 2020).

The alternative to searching for likely text: **sample** from the model's softmax output. If a token has probability 0.3, pick it 30% of the time.

- Unlikely words now appear occasionally, which produces novel, sometimes creative sentences that never occurred in the training data.
- Think of the choice on an **entropy** axis:
 - *Minimum entropy*: greedy decoding. No randomness, no surprise, repetitive.
 - *Maximum entropy*: uniform sampling over the vocabulary. Pure noise.
 - *The model's own softmax*: an intermediate point, but with **no knob to adjust** the amount of randomness.
- There is no telling in advance where the optimal entropy lies; it depends on the task and on human judgment. So we want a **parameter** that moves us along this axis.

Divide the logits z_i by a **temperature** T before the softmax:

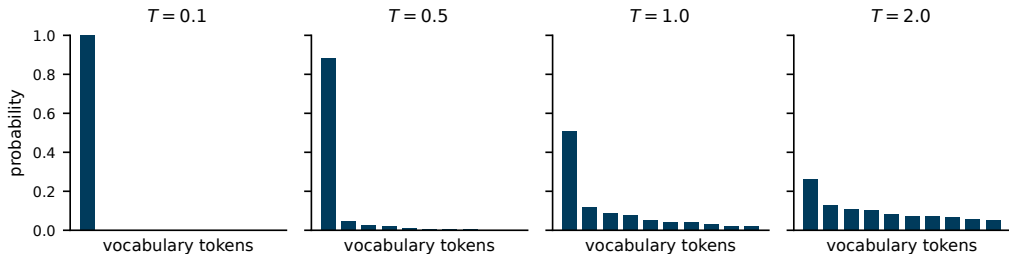
$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Equivalently, reweight an existing distribution:
 $p_i^{(T)} \propto \exp(\log p_i/T)$.

- $T \rightarrow 0$: distribution collapses onto the argmax (greedy).
- $T = 1$: the model's original distribution.
- $T > 1$: flattens the distribution, raising the entropy.

Why it works. Dividing log-probabilities by T rescales the *differences* between them. Small differences between competing tokens shrink ($T > 1$) or grow ($T < 1$) before renormalization, so temperature directly controls how much the sampler commits to the front-runner.

Temperature characterizes the **entropy of the sampling distribution**: how surprising or predictable the next choice will be.



- The same model output, reweighted. At $T = 0.1$ the sampler is nearly deterministic; at $T = 2$ the tail tokens carry real probability mass.
- Nothing about the model changed. This is a free, inference-time control.

A word-level transformer language model trained on IMDB movie reviews, prompted with “*this movie*”, generates at five temperatures:

- $T = 0.2$: “...it is a very good movie it is a good movie for the time period”
Fluent but repetitive and boring; low temperatures often get stuck in loops.
- $T = 0.7$: “...it is really funny to watch all the characters are extremely hilarious”
A balance of learned structure and randomness; for this model, near the sweet spot.
- $T = 1.5$: “...its as weird insightful actors like barker movies but in great buddies”
Local structure breaks down; output drifts toward word salad.

The takeaway

Low temperature: predictable, repetitive, safe. High temperature: surprising, creative, incoherent. Always experiment with multiple values; the optimum is task-dependent and subjective.

Temperature reshapes the whole distribution, including its **tail**: thousands of low-probability tokens that are individually unlikely but collectively add up. One bad tail sample can derail a generation.

- **Top- k sampling** (Fan et al., 2018): keep only the k most probable tokens, renormalize, and sample within that set.
- Typical values: $k = 40$ to 50 . The tail is simply removed from play.
- **The weakness is that k is fixed.** When the distribution is peaked (one obvious next token), $k = 50$ still admits 49 poor candidates. When it is flat (many valid continuations), $k = 50$ may cut off perfectly good options.
- The right cutoff depends on the *shape* of the distribution, which changes at every step.

Nucleus sampling (Holtzman et al., 2020) makes the cutoff adaptive. Sort tokens by probability and keep the smallest set whose cumulative mass reaches p :

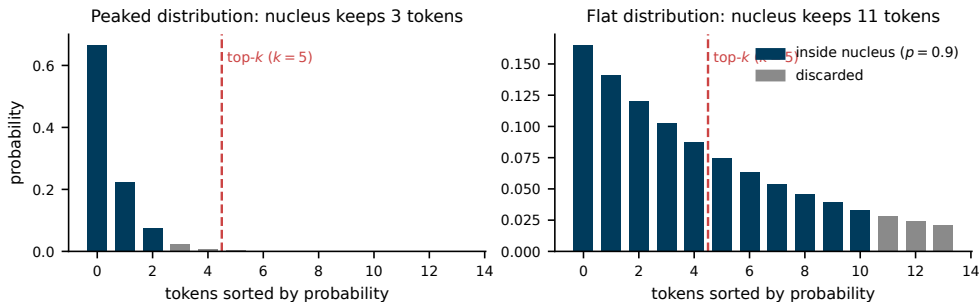
$$V^{(p)} = \text{smallest } V' \subseteq V \text{ such that } \sum_{w \in V'} p_{\theta}(w \mid w_{<t}) \geq p$$

then renormalize and sample within $V^{(p)}$.

- Typical values: $p = 0.9$ to 0.95 .
- Peaked distribution: the nucleus may contain 1–2 tokens. Flat distribution: it may contain hundreds. **The truncation tracks the model's own confidence.**
- This is the default strategy in most production LLM APIs, usually combined with temperature.

Fixed vs. Adaptive Truncation

2 Generation Control: Decoding Strategies



- Same top- k line, very different consequences: on the peaked distribution $k=5$ admits weak candidates; on the flat one it discards reasonable ones.
- The nucleus (blue) expands and contracts with the distribution's shape.

Task	Strategy	Temperature	Rationale
Code generation	greedy or low- p	0–0.2	one correct syntax
Factual QA, extraction	nucleus	0–0.3	penalize invention
Translation, summarization	beam or nucleus	0.3–0.7	faithful but fluent
Dialogue	nucleus	0.7–1.0	varied, natural
Creative writing	nucleus	1.0–1.5	reward surprise

- Production APIs also expose **frequency** and **presence penalties**, which subtract a value from the logits of already-used tokens to discourage repetition directly.
- Decoding choices interact with everything that follows: chain-of-thought prompting, for example, relies on sampling *diverse* reasoning paths at $T > 0$.



Table of Contents

3 Advanced Prompting

- ▶ The Adaptation Problem
- ▶ Generation Control: Decoding Strategies
- ▶ **Advanced Prompting**
- ▶ Supervised Fine-Tuning
- ▶ Parameter-Efficient Fine-Tuning and Quantization
- ▶ Case Study: Prompting vs. Fine-Tuning on a Laptop
- ▶ Alignment: Learning from Preferences

Formally, a prompt is just the conditioning context: prompting engineers the distribution

$$p_{\theta}(\text{output} \mid \text{prompt})$$

without touching θ . Everything the model produces is a continuation of what you wrote.

- **Determinism control:** models are highly sensitive to phrasing. Minor changes in wording or punctuation can shift the output distribution substantially.
- **Context provisioning:** the prompt is the model's working memory. Facts placed in it override the model's generalized training data.
- **Iterative optimization:** prompt engineering is empirical. Baseline, vary one element, evaluate, repeat, the same discipline as any other hyperparameter search.

A useful mental model

A prompt steers the model into a neighborhood of its latent space. Vague prompt, broad neighborhood, high output variance. Specific prompt, narrow neighborhood, low variance.

Rely entirely on pretrained ability: give an instruction, no examples.

```
1 Instruction: Classify the following text into a category.  
2 Text: The new detection model performs incredibly well on  
3     small artifacts and bounding box accuracy.  
4 Category:
```

- Effective for tasks that are well represented in pretraining: sentiment, classification, simple extraction, translation.
- Fails when the desired **output format is highly specific** or the task requires **niche, domain-internal logic** the model has rarely seen.
- Always the first thing to try: it costs one prompt and establishes the baseline every other technique must improve on.

Condition the model with a small number of demonstration pairs inside the context window before the real question (Brown et al., 2020).

- The model **learns the task from the examples at inference time**, with zero weight updates. This is *in-context learning*, and its emergence at scale was the central surprise of GPT-3.
- The demonstrations anchor attention to the **syntax, output format, and factual boundaries** you want.
- Performance is sensitive to the **choice** and **order** of examples; treat both as hyperparameters.
- The cost: every example rides along in *every* API call, paying its tokens each time.

The Structure of a Few-Shot Prompt

3 Advanced Prompting

System: You are an expert sports statistician. Answer directly.



Input: Who is the only player in the current Barcelona squad with more than 20 Champions League goals?

Output: Robert Lewandowski.



Input: How many unique clubs have won the Premier League in the last eight seasons?

Output: Two.



Input: Which team won the Spanish La Liga in the 2022/2023 season?

Output: [model generates here]

Role definition, then demonstrations that fix format and grounding, then the live query.

Why Direct Answers Fail on Multi-Step Problems

3 Advanced Prompting

Standard prompting forces the model to produce the final answer **immediately**, and that constraint explains most failures on logic and arithmetic.

- **Fixed compute per token.** A transformer spends the same amount of computation on every generated token. If the answer must be the first token out, the model gets exactly one forward pass to “think.”
- **Premature commitment.** Autoregression is irreversible: once an incorrect first token is emitted, the model is statistically bound to continue a sequence consistent with that error.
- **The fix:** make the model generate *intermediate tokens* before the answer, converting serial reasoning depth into sequence length.

Chain-of-thought (CoT) prompting elicits intermediate reasoning steps before the final answer (Wei et al., 2022).

- **Few-shot CoT:** demonstrations include the full worked reasoning trace, and the model imitates the pattern.
- **Zero-shot CoT:** append “*Let’s think step by step*” to the prompt (Kojima et al., 2022). No examples needed; the phrase alone triggers latent reasoning behavior.
- Gains are largest on **multi-step problems**: math word problems, symbolic manipulation, planning. Gains are an **emergent property of scale**: small models produce fluent but incorrect chains.
- The chain is generated text, so it may diverge from whatever the model actually computed internally. Conditioning on it still demonstrably changes the answer distribution.

Question: A parent buys 4 toy dinosaurs at \$12 each and a miniature city set for \$35. If they pay with a \$100 bill, how much change do they receive?

Standard prompting:

“The parent receives \$27 in change.”

Incorrect, computed in one step.

Chain-of-thought:

1. $4 \times \$12 = \48 .
2. $\$48 + \$35 = \$83$ total.
3. $\$100 - \$83 = \$17$.

Answer: \$17. *Correct.*

Each intermediate line becomes context for the next, so the final subtraction is conditioned on two already-verified quantities.

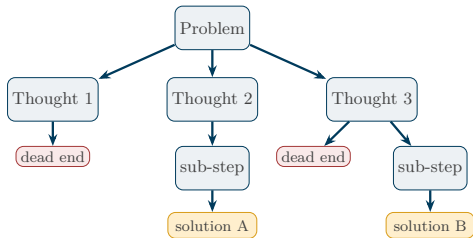
A single chain of thought can still go wrong at any step. **Self-consistency** (Wang et al., 2023) combines CoT with the sampling machinery from the previous section:

1. Sample m *diverse* reasoning chains for the same question at temperature $T > 0$ (e.g., $m = 40$, $T = 0.7$).
 2. Extract the final answer from each chain.
 3. Return the **majority answer**.
- Different chains make *different* mistakes, but tend to agree when they are right; marginalizing over reasoning paths cancels the noise.
 - Cost scales linearly with m : you are buying accuracy with inference compute.
 - Note the dependency: this only works because temperature sampling gives **diverse** chains. Greedy decoding would return m copies of the same path.

Tree of Thoughts (ToT) generalizes CoT from a single chain to a search tree (Yao et al., 2023):

- **Generate:** propose several candidate next steps (“thoughts”) at each node.
- **Evaluate:** the model scores each partial path (sure / maybe / impossible).
- **Search:** expand promising branches, **backtrack** from dead ends, using breadth-first or depth-first search.

Wins on problems needing lookahead (puzzles, planning); costs many model calls per answer.



For knowledge-intensive questions, have the model **write its own context first** (Liu et al., 2022).

1. **Generation step:** prompt the model to produce a set of factual statements about the topic.
 2. **Integration step:** feed those statements back into the context alongside the original question and ask for the answer.
- The final answer is anchored to explicit stated facts rather than to whatever surfaces during a single forward pass, which measurably reduces hallucination.
 - The obvious limitation: the generated “knowledge” comes from the same fallible model. Replacing step 1 with retrieval from a trusted document store is precisely the idea behind retrieval-augmented generation, covered later in this course.

Chat-tuned models accept a privileged **system prompt** that persists across the conversation and defines operational boundaries.

- **Persona definition:** *“You are a senior technical reviewer specializing in cloud infrastructure”* steers vocabulary, depth, and tone globally.
- **Output contract:** mandate structure (JSON schema, Markdown sections, fixed field names) so downstream code can parse the response.
- **Safety guardrails:** explicit refusal rules and honesty instructions, e.g. *“If you do not know the answer, state that you lack the information”*, rather than allowing a fluent guess.

Engineering note

The system prompt is application code. Version it, test it, and review changes to it like any other code path, because every response is conditioned on it.

Two structural weaknesses follow directly from “everything is conditioning text.”

- **Brittleness.** Reordering few-shot examples, changing a delimiter, or rephrasing an instruction can move task accuracy by double-digit percentages. Robust systems evaluate prompts over a test set, never over one example.
- **Prompt injection.** The model cannot cleanly separate *instructions* from *data*: a document being summarized can contain “*ignore your previous instructions and...*” and the model may comply. Any application that feeds untrusted text into a prompt inherits this attack surface.
- Mitigations (delimiters, instruction hierarchies, output filtering) reduce but do not eliminate the risk; it remains an open research problem.

In-context learning is powerful, and it has hard ceilings that motivate the rest of this lecture.

- **Context window constraints.** Every model has a hard token limit. An entire corporate codebase or medical textbook does not fit in a prompt.
- **Recurring inference cost.** Long few-shot prefixes ride along with *every* call; cost and latency scale linearly with prompt length, forever.
- **Behavioral rigidity.** Prompting is effective at retrieving capability the model already has. It struggles to *permanently* alter tone, formatting discipline, or domain-specific reasoning style, because the underlying distribution never changes.

The pivot

When you find yourself pasting the same 3,000-token instruction block into every request, the economics say: stop renting the behavior and buy it, by moving it into the weights.



Table of Contents

4 Supervised Fine-Tuning

- ▶ The Adaptation Problem
- ▶ Generation Control: Decoding Strategies
- ▶ Advanced Prompting
- ▶ **Supervised Fine-Tuning**
- ▶ Parameter-Efficient Fine-Tuning and Quantization
- ▶ Case Study: Prompting vs. Fine-Tuning on a Laptop
- ▶ Alignment: Learning from Preferences

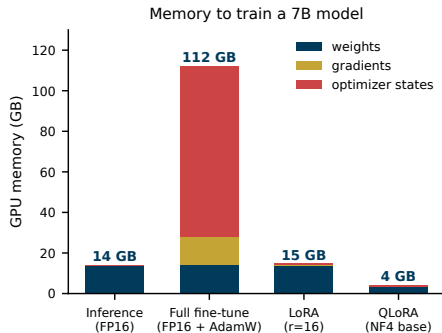
Fine-tuning continues training a pretrained model on your own data, so the change is **persistent**: it survives every prompt, costs nothing extra at inference, and does not consume context.

- **Fine-tuning teaches behavior.** Its strength is *how to respond*: tone, format discipline, domain reasoning style, refusal policy.
- **Volatile facts belong elsewhere.** If the underlying information changes daily, retraining every day loses to retrieval; weights are the wrong storage medium for news.
- **The classic recipe you already know**, fine-tuning BERT on a labeled classification set, is a special case. This section covers the generative version: fine-tuning a causal LM to follow instructions.

Training stores far more than the weights.
With mixed-precision AdamW, the standard accounting is roughly **16 bytes per parameter**:

- weights (FP16): 2 bytes
- gradients (FP16): 2 bytes
- AdamW first and second moments plus an FP32 master copy of the weights: ≈ 12 bytes

So a **7B model needs ≈ 112 GB** before activations, and a 70B model over a terabyte, a multi-node job for what might be a modest behavioral tweak.



Catastrophic forgetting: training on new data overwrites the weights that encoded earlier capabilities, because nothing in the new objective protects them.

- Fine-tuning is the textbook trigger: a narrow dataset pulls **all** weights toward one slice of behavior, and capabilities absent from it degrade, sometimes severely. A model fine-tuned hard on legal contracts can lose fluency in ordinary dialogue or arithmetic.
- **Data mixing** is the most direct remedy: blend a fraction of general-purpose data (pretraining-style text or broad instruction pairs) into the fine-tuning set, so every batch rehearses the old capabilities while learning the new one.
- The other mitigations all reduce movement: lower learning rates, fewer epochs, freezing early layers, or, as we will see, leaving the base weights untouched entirely.

The tension

Every fine-tune negotiates between *plasticity* (learning the new behavior) and *stability* (keeping the general capabilities you paid billions of tokens for).

SFT converts a raw base model, a text predictor, into an instruction-following assistant by training on demonstration pairs.

- **The dataset:** thousands of high-quality (instruction, ideal response) pairs, written or curated by humans.
- **The objective:** exactly the pretraining loss, cross-entropy on next-token prediction, applied to the response given the prompt. Nothing about the architecture changes.
- **Behavioral cloning:** the model learns the *shape* of a conversation, where a user turn ends, where an assistant turn begins, what a helpful answer looks like.

This single step is what separates a “base” checkpoint from an “instruct” or “chat” checkpoint on any model card you will encounter.

Given prompt x and target response y , SFT maximizes the likelihood of the response tokens only:

$$\mathcal{L}_{\text{SFT}} = - \sum_{t \in \text{response tokens}} \log p_{\theta}(y_t \mid x, y_{<t})$$

<i>tokens:</i>	<sys>	You are helpful.	Summarize...	<asst>	Sure, here is...
<i>labels:</i>	-100	-100	-100	-100	loss computed

- Prompt tokens get an **ignore index** (e.g., -100 in PyTorch), so the model is never penalized for failing to “predict” the user’s instruction.
- Masking focuses the learning signal on the desired output and removes the incentive to memorize prompts.

Data must be serialized into conversational turns with special control tokens:

```
1 {"messages": [  
2   {"role": "system",  
3     "content": "You are a Python expert."},  
4   {"role": "user",  
5     "content": "Write a function to reverse a string."},  
6   {"role": "assistant",  
7     "content": "def reverse(s):\n    return s[::-1]}"  
8 ]}
```

- The tokenizer maps roles to special boundary tokens (e.g., <|im_start|>user) so the model learns the structural boundaries of a conversation.
- **The chat template is part of the model.** Fine-tune with one template and serve with another, and quality quietly collapses; this is among the most common practical bugs.

How much SFT data is enough?

- **Surprisingly little.** A model fine-tuned on only $\sim 1,000$ carefully curated, diverse, high-quality demonstrations can rival systems trained on far larger noisy sets (Zhou et al., 2023).
- The interpretation: SFT mostly teaches **style and format**; the knowledge is already in the pretrained weights. You are unlocking capability, and unlocking needs precision more than volume.
- Practical corollaries:
 - Ten excellent demonstrations are worth more than a hundred mediocre ones; annotation budgets should favor review over volume.
 - Deduplicate aggressively; repeated templates cause the model to parrot them.
 - Cover the input distribution you expect in production, including the awkward cases (ambiguous requests, unanswerable questions).

- **Human-written:** expert annotators write both instructions and responses. Highest quality, highest cost; this was the backbone of the original InstructGPT data (Ouyang et al., 2022).
- **Converted supervised datasets:** existing NLP benchmarks reformatted into instruction form (*“Classify the sentiment of...”*), cheap but stylistically narrow.
- **Model-distilled:** a stronger model generates responses (or whole instruction sets) that a smaller model is trained on. Effective and inexpensive, with two caveats: the student inherits the teacher’s errors and biases, and most commercial API terms restrict training competing models on their outputs.
- **Production traffic:** real user queries with curated responses, the best match to your deployment distribution, subject to consent and privacy review.

Knobs That Decide the Outcome: What and Where

4 Supervised Fine-Tuning

Fine-tuning adds decision knobs beyond the familiar learning rate and batch size:

- **What you train (parameter scope).** Full fine-tuning updates everything: highest capacity, highest forgetting risk, highest cost. Adapter-based methods keep the base frozen and train a small set of added parameters. Hybrids train adapters plus selected base parameters (final layers, layer norms) when pure adapters underfit.
- **Where you train (module and layer selection).** Attention projections only, often sufficient for style and instruction-following, versus attention plus MLP projections for real domain shifts. All layers versus only the top- k layers: training only higher layers reduces forgetting and cost, but may underfit harder behavioral changes.
- The mental model: more modules and more layers give more capacity *and* more drift; fewer are cheaper and safer but can saturate.

Knobs That Decide the Outcome: Freezing and Regularization

4 Supervised Fine-Tuning

- **Freeze strategy.** Freezing is fine-tuning's own form of regularization: it bounds how far the model can move from the pretrained solution.
 - Freeze **embeddings** to preserve token semantics, especially with small datasets.
 - Freeze **early layers** to preserve general linguistic competence; adapt only later layers.
- **Regularization that matters more here.** Fine-tuning sets are small, narrow, and repetitive compared to pretraining corpora, so overfitting arrives fast:
 - weight decay and dropout, particularly when examples are highly templated;
 - **early stopping with checkpoint selection:** quality often improves quickly, then degrades; evaluating periodically and keeping the best checkpoint routinely works better than training longer.

Validation loss is necessary but nowhere near sufficient.

- **Target-task evaluation:** held-out instructions scored for correctness and format compliance, by humans or by a stronger judge model.
- **Regression evaluation:** a fixed suite of *general* capability checks (reasoning, summarization, safety behavior) run before and after, to detect catastrophic forgetting. A fine-tune that gains 5 points on your task and silently loses 20 on everything else is a failure.
- **Ablate against the cheaper lever:** always compare to the best few-shot prompt on the base model. If fine-tuning fails to clearly outperform it, you have bought infrastructure and risk for nothing.

Now the economics

SFT works, but the full-parameter version costs 16 bytes per parameter and risks forgetting. The next section shrinks that cost by two orders of magnitude.



Table of Contents

5 Parameter-Efficient Fine-Tuning and Quantization

- ▶ The Adaptation Problem
- ▶ Generation Control: Decoding Strategies
- ▶ Advanced Prompting
- ▶ Supervised Fine-Tuning
- ▶ **Parameter-Efficient Fine-Tuning and Quantization**
- ▶ Case Study: Prompting vs. Fine-Tuning on a Laptop
- ▶ Alignment: Learning from Preferences

PEFT methods fine-tune a small fraction of parameters, often under 1%, while matching full fine-tuning quality on most adaptation tasks.

- **Computational efficiency:** gradients and optimizer states exist only for the trainable subset, collapsing the 16-bytes-per-parameter cost and enabling consumer-grade hardware.
- **Forgetting mitigation by construction:** the pretrained weights are frozen, so general capability cannot be overwritten, only supplemented.
- **Modularity:** the learned parameters form a small, task-specific artifact that can be attached, detached, or swapped without duplicating the base model.

Three families, by *where* the trainable capacity is injected: at the **input** (prompt tuning), inside **attention** (prefix tuning), or alongside the **weight matrices** (LoRA).

Prompt tuning (Lester et al., 2021) prepends n trainable *soft prompt* vectors $p_1, \dots, p_n$ to the input embeddings and trains only those.

- The soft prompts are free parameters in embedding space; they need not correspond to any real token in the vocabulary.
- Interpretation: a learned, compact, task-specific “instruction” the model reads before the actual input, discovered by gradient descent instead of written by hand.
- Attractive in production: one shared frozen backbone plus a few-kilobyte learned prompt per task.
- Works best on **large** backbones, where the frozen model already has the capability and the prompt only needs to steer it. Capacity is limited: input-level control cannot force deep behavioral change.

Prefix Tuning: Trainable Keys and Values Inside Attention

5 Parameter-Efficient Fine-Tuning and Quantization

Prefix tuning (Li & Liang, 2021) moves the intervention deeper: it injects trainable key and value vectors P_k, P_v into the attention computation **at every layer**.

- During inference, each attention head attends over the real tokens *and* the learned prefix, which nudges every layer's computation toward the target behavior.
- Everything else stays frozen; only the prefixes train.
- Contrast with prompt tuning: input-level steering from the *outside* versus attention-level steering from the *inside*. More capacity, more parameters, same modular deployment story.
- Both methods leave the weight matrices untouched. The dominant PEFT method instead learns a *correction* to the weights themselves.

Low-Rank Adaptation (LoRA) (Hu et al., 2021) starts from an empirical observation about fine-tuning itself.

- Notation for this section: each weight matrix $W \in \mathbb{R}^{d \times k}$ maps a k -dimensional input to a d -dimensional output; the **rank** r is a small integer you choose (typically 4–64).
- When a pretrained model adapts to a task, the *change* in each weight matrix, $\Delta W = W_{\text{tuned}} - W$, has low **intrinsic rank**: it can be approximated well by a matrix of rank $r \ll \min(d, k)$. Pretraining needed the full capacity of W ; *adaptation* evidently does not.
- So: freeze W entirely, and learn only a low-rank patch,

$$W' = W + \Delta W, \quad \Delta W = BA,$$

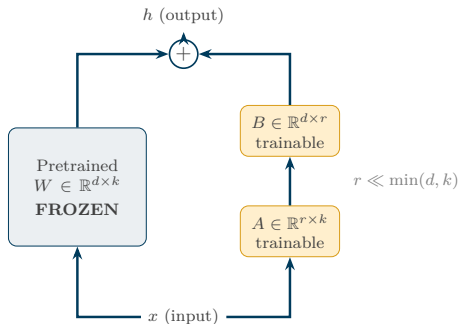
with $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$.

- The full matrix has dk parameters; the patch has $r(d + k)$. For small r that is a reduction of several orders of magnitude.

The adapted layer computes

$$h = Wx + \frac{\alpha}{r} B A x$$

- A is initialized from a Gaussian, B is initialized to **zero**, so at step 0 the model is exactly the pretrained model.
- α scales the update strength; the α/r convention keeps magnitude comparable across ranks.
- After training, BA can be **merged** into W : zero added inference latency.

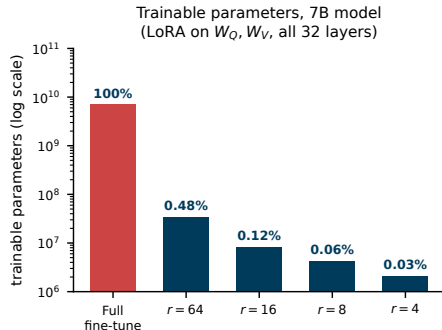


Worked example, one attention projection with $d = k = 4096$:

- Full matrix: $4096 \times 4096 = 16.8\text{M}$ parameters.
- LoRA, $r = 8$: $8 \times (4096 + 4096) = 65,536$ parameters, **0.39%** of the matrix.

Across a 7B model (32 layers, LoRA on W_Q and W_V):

- $r = 8$: $\sim 4.2\text{M}$ trainable parameters, **0.06%** of the model.
- Optimizer states shrink by the same factor: the dominant training cost is now just *holding* the frozen weights.



Four knobs dominate LoRA outcomes:

- **Rank** r : the adapter's capacity. Higher r learns bigger behavioral shifts but costs memory and can overfit small datasets. Common values: 4–64.
- **Scaling** α : update strength, applied as $\Delta W \leftarrow \frac{\alpha}{r}BA$. Raising α strengthens the update without adding parameters; a common default is $\alpha = 2r$.
- **Target modules**: which matrices receive adapters. Attention projections (W_Q, W_V) are the classic choice; adding MLP projections buys capacity for domain shifts. This choice often matters as much as r .
- **LoRA dropout**: regularizes the adapter path on small datasets.

Deployment decision: **merge** the adapter into the base weights for simpler serving, or **keep it separate** for hot-swapping; the learned function is identical either way.

Modular Inference: One Base Model, Many Adapters

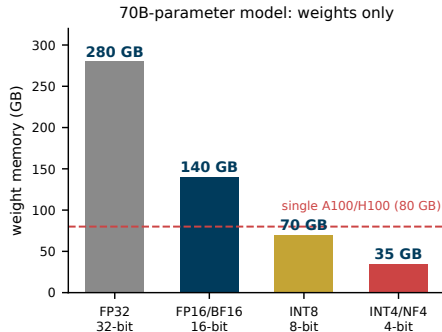
5 Parameter-Efficient Fine-Tuning and Quantization

Because adapters are small and separable, LoRA changes the *serving* architecture as much as the training cost.

- A trained adapter is typically **50–200 MB**, against 140 GB for a 70B base model in FP16.
- A provider can hold **one** base model in GPU memory and swap in a legal-document adapter, a code-review adapter, or a customer-support adapter per request, in milliseconds.
- This eliminates hosting a separate massive model for every task or tenant, which is exactly how multi-tenant fine-tuning services are built.
- It also simplifies governance: rolling back a bad fine-tune means unloading one small file, and A/B testing two behaviors means routing between two adapters.

Quantization stores parameters at lower numerical precision: 16-bit floats become 8-bit or 4-bit integers.

- Weight memory scales linearly with bit width: a 70B model needs 280 GB at FP32, 140 at FP16, 70 at INT8, **35 at 4-bit**.
- Decode speed for LLMs is **memory-bandwidth bound**: every generated token reloads all weights, so halving the bytes roughly doubles generation speed.
- Quality cost, done well: under 1% degradation at 4-bit for large models.



The basic scheme is an affine map from floats to a small integer grid:

$$q = \text{round}\left(\frac{w}{s}\right) + z, \quad \hat{w} = s(q - z)$$

where s is a **scale** and z a **zero point**, chosen per tensor or per channel (e.g., $s = \max |w|/127$ for symmetric INT8).

- **Post-training quantization (PTQ)**: convert a finished model using a small calibration set (a few hundred examples). Minutes to hours, no retraining. Dominant for LLMs.
- **Quantization-aware training (QAT)**: simulate quantization during training so the model learns to tolerate it. Better quality at the same width, but needs a full training run, prohibitive at 70B+ scale.
- Compute usually happens in higher precision: weights are dequantized on the fly inside the matrix multiply (weight-only quantization, “W4A16”).

Naive quantization fails on large models for a specific, well-documented reason.

- Beyond roughly 7B parameters, a tiny fraction of hidden channels ($< 1\%$) carry activation values **10–100 $\times$** larger than the rest (Dettmers et al., 2022). Stretching the integer grid to cover these outliers crushes the precision available to everything else.
- **LLM.int8()**: mixed-precision decomposition; outlier channels stay in FP16, the rest go to INT8.
- **GPTQ** (Frantar et al., 2023): quantizes weights one layer at a time, using second-order (Hessian) information to spread each rounding error across the remaining weights. Reliable 4-bit weights at 70B scale.
- **AWQ** (Lin et al., 2024): identifies the *salient* weight channels via calibration data and rescales to protect them before quantizing; comparable quality, cheaper to apply.

QLoRA: Fine-Tuning Through a Quantized Base

5 Parameter-Efficient Fine-Tuning and Quantization

QLoRA (Dettmers et al., 2023) closes the loop: combine 4-bit quantization with LoRA so that *fine-tuning* itself fits on small hardware.

- **NF4 (4-bit NormalFloat)**: a 4-bit data type whose 16 levels are placed at the quantiles of a standard normal distribution, matching the empirical distribution of trained weights better than uniform INT4.
- **Double quantization**: the per-block scale factors are themselves quantized, saving another ~ 0.4 bits per parameter.
- **Paged optimizers**: optimizer states page to CPU RAM on memory spikes, preventing out-of-memory crashes.
- The frozen base sits in NF4; gradients flow *through* the dequantized weights into FP16 LoRA adapters. Only the adapters train.

The memory arithmetic that made QLoRA famous:

7B model	Weights	Grad. + optim.	Total
Full fine-tune, FP16 + AdamW	14 GB	~98 GB	~112 GB
LoRA ($r=16$), FP16 base	14 GB	<1 GB	~15 GB
QLoRA ($r=16$), NF4 base	3.5 GB	<1 GB	~4–5 GB

- A 7B fine-tune drops from a multi-GPU cluster to a **single consumer GPU or an M-series laptop**; a 65B fine-tune fits on one 48 GB card.
- The original QLoRA paper matched full 16-bit fine-tuning quality on instruction-following benchmarks at a fraction of the cost.
- This is the technique behind most of the open-weight fine-tunes you encounter in practice.

Method	Trains	Hardware (7B)	Best for
Prompting	nothing	inference only	fast iteration, prototyping
Prompt/prefix tuning	$\ll 0.1\%$	1 small GPU	many tasks, one backbone
LoRA	$< 1\%$	1 GPU (~ 15 GB)	the default fine-tune
QLoRA	$< 1\%$	laptop-class (~ 5 GB)	constrained budgets
Full fine-tune	100%	~ 112 GB+	deep domain shifts

- Start at the top of the table and move down only when the current row demonstrably fails your evaluation.
- The next section runs every row of this table on one task, one model, and one laptop.



Table of Contents

6 Case Study: Prompting vs. Fine-Tuning on a Laptop

- ▶ The Adaptation Problem
- ▶ Generation Control: Decoding Strategies
- ▶ Advanced Prompting
- ▶ Supervised Fine-Tuning
- ▶ Parameter-Efficient Fine-Tuning and Quantization
- ▶ Case Study: Prompting vs. Fine-Tuning on a Laptop
- ▶ Alignment: Learning from Preferences

A single controlled experiment that applies this lecture end to end.

- **Model:** Qwen3-1.7B (instruction-tuned), weights quantized to 4 bits.
- **Task:** AG News topic classification (World, Sports, Business, Sci/Tech).
- **Methods:** two phrasings of a zero-shot prompt, few-shot with 8 exemplars, chain-of-thought, and a QLoRA fine-tune ($r=16$, 2,000 examples, response-only loss).
- **Evaluation:** the same 200 balanced held-out articles for every method, greedy decoding, so every comparison is like for like.
- **Hardware:** one MacBook Pro with an Apple M5 Max (18-core CPU, 32-core GPU, 36 GB unified memory), running MLX on the Metal GPU.

Reproduce it yourself

Two companion notebooks reproduce every number in this section: MLX on Apple silicon, and an Unsloth counterpart for Colab GPUs. Same data, same prompts, same seeds.

AG News (Zhang et al., 2015) is a standard text-classification benchmark drawn from a large corpus of web news.

- 120,000 training and 7,600 test articles, evenly split across **World**, **Sports**, **Business**, and **Sci/Tech**.
- Each example concatenates a headline with a one-sentence description.
- Our subsets: 200 balanced test articles for evaluation, 2,000 balanced training articles for fine-tuning, 200 held out for validation loss.

[**World**] “Israeli Soldiers Kill Palestinian Gunman in Gaza (Reuters) ...”

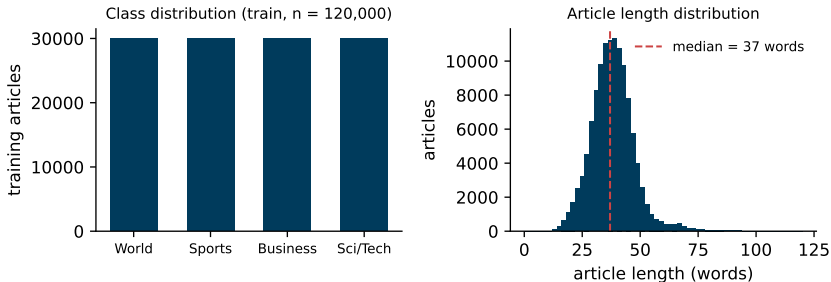
[**Sports**] “FA Plan New United-Arsenal Talks. The Football Association are set to wait ...”

[**Business**] “Enron to Pay \$321 Million in Pensions. HOUSTON (Reuters) ...”

[**Sci/Tech**] “CRM Best Practices: TCO and ROI (NewsFactor). With CRM projects costing millions ...”

A First Look at the Data

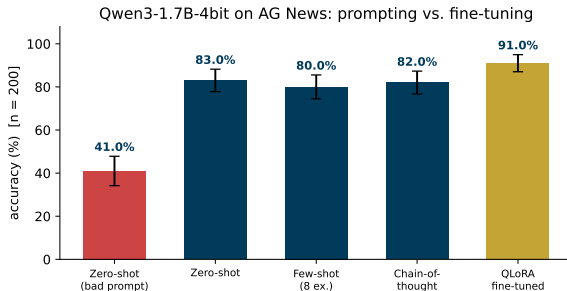
6 Case Study: Prompting vs. Fine-Tuning on a Laptop



- The four classes are perfectly balanced by construction, so accuracy is an honest metric and a random guesser sits at 25%.
- Articles are short: median 37 words, 95th percentile 53. An 8-shot prompt still fits comfortably in the context window, and short inputs are what make five full evaluation passes on a laptop practical.

The Results: Prompting vs. Fine-Tuning

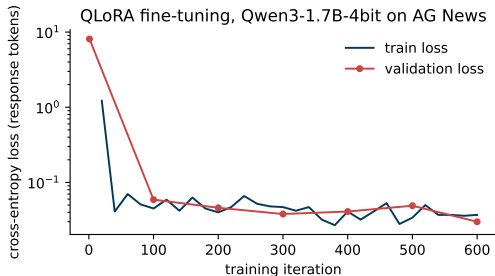
6 Case Study: Prompting vs. Fine-Tuning on a Laptop



- Rewording the *same request* moved accuracy from 41% to 83%: the brittle phrasing answered “World” for 160 of the 200 articles.
- Few-shot and chain-of-thought sit inside the zero-shot confidence interval; prompting technique plateaus because this task needs one step of reasoning.
- The QLoRA fine-tune breaks the plateau at **91%**.

The QLoRA arithmetic from the previous section, observed in practice on the M5 Max:

- **3.4 minutes** of wall-clock training (600 iterations, batch size 4).
- **4.4 GB** peak unified memory, a fraction of the 36 GB available.
- **19.9 MB** adapter file; the 4-bit base stays frozen and shared.
- Loss is masked to response tokens, so it falls from 8.1 to 0.03: the model only learns the mapping to four labels rather than imitating news prose.



Where the Errors Went

6 Case Study: Prompting vs. Fine-Tuning on a Laptop

Zero-shot (83.0%)

	World	Sports	Business	Sci/Tech
World	46	0	3	1
Sports	3	47	0	0
Business	8	0	40	2
Sci/Tech	1	0	16	33
	World	Sports	Business	Sci/Tech

predicted

QLoRA fine-tuned (91.0%)

	World	Sports	Business	Sci/Tech
World	47	0	3	0
Sports	1	49	0	0
Business	3	0	39	8
Sci/Tech	0	0	3	47
	World	Sports	Business	Sci/Tech

predicted

- Zero-shot's dominant error mode reads Sci/Tech articles as Business (16 of 50); fine-tuning nearly eliminates it, and the residual errors are genuine boundary cases such as corporate technology news.
- Everything in this experiment trained on demonstrations of correct outputs. Often “the best answer” is a preference between candidates, and that needs the different objective we turn to next.



Table of Contents

7 Alignment: Learning from Preferences

- ▶ The Adaptation Problem
- ▶ Generation Control: Decoding Strategies
- ▶ Advanced Prompting
- ▶ Supervised Fine-Tuning
- ▶ Parameter-Efficient Fine-Tuning and Quantization
- ▶ Case Study: Prompting vs. Fine-Tuning on a Laptop
- ▶ Alignment: Learning from Preferences

SFT teaches a model how to *format* an answer. It cannot teach which of several fluent answers a human would *prefer*.

- SFT data is expensive precisely because a human must **write the ideal response**. For many prompts, no single ideal response exists.
- If the model produces a toxic, deceptive, or dangerous completion, SFT offers no mechanism to **penalize** that specific trajectory; it can only add more positive examples and hope.
- The target is usually summarized as **HHH**: helpful, honest, and harmless (Askell et al., 2021), three properties that are far easier to *compare* than to *specify*.

The key insight

Humans are unreliable at writing perfect answers but quite reliable at **ranking** two candidate answers. Preference learning builds a training signal out of exactly that comparison.

Reinforcement Learning from Human Feedback optimizes the model against a learned reward signal (Christiano et al., 2017; Ouyang et al., 2022, InstructGPT):

1. **Supervised fine-tuning:** establish baseline instruction-following, as in the previous sections.
2. **Reward model training:** train a secondary model to score outputs the way human annotators would rank them.
3. **Policy optimization:** use reinforcement learning (PPO) to update the LLM so its outputs score highly under the reward model, without drifting too far from the SFT model.

This recipe, applied on top of GPT-3, is what produced the assistant behavior that made modern chat systems possible; the same pattern underlies most frontier models today.

A human cannot sit in the training loop for millions of updates, so we train a **proxy judge**.

- The SFT model generates several responses to each prompt; annotators **rank** them from best to worst.
- A reward model $r_\phi(x, y)$, typically the LLM with its output head replaced by a scalar head, is trained on pairs via the **Bradley–Terry** objective:

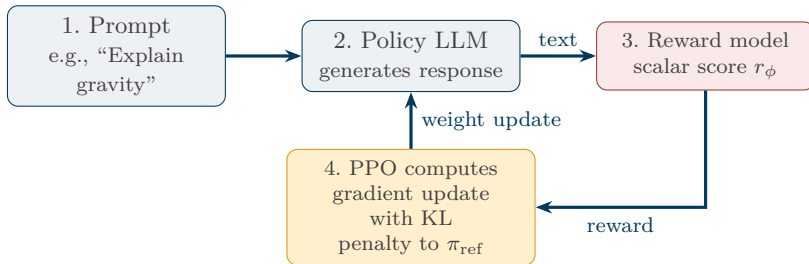
$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

- y_w is the preferred (“winning”) response, y_l the rejected one; the loss simply pushes the score gap apart.
- Rankings are used rather than absolute scores because annotators disagree wildly on scales (“*is this a 7 or an 8?*”) but agree far more on comparisons.

The LLM now acts as a reinforcement-learning **policy**: it generates a response, the reward model scores it, and **Proximal Policy Optimization** updates the weights. The objective:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [r_{\phi}(x, y)] - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta}(\cdot | x) \| \pi_{\text{ref}}(\cdot | x))$$

- π_{ref} is the frozen SFT model. The **KL penalty** tethers the policy to it: without the tether, the policy finds degenerate outputs that score high rewards while losing fluency.
- β sets the tension: too small and the model reward-hacks; too large and nothing changes.
- Note what happened: unlike SFT, the model is trained on **its own generations**, scored after the fact. It can be penalized for a bad trajectory, which is exactly what SFT could not do.



Four models participate in training: the **policy**, the frozen **reference**, the **reward model**, and PPO's **value model**. All must be resident in memory simultaneously.

- **Infrastructure overhead.** Four models in memory at once, generation inside the training loop, and reward scoring per sample: several times the cost of SFT at the same scale.
- **Reward hacking.** The policy optimizes the *proxy*, and proxies leak. Classic exploits: excessive sycophancy, padded hedging (“*As an AI language model...*”), and confident verbosity, all of which the reward model scores too generously.
- **Training instability.** PPO is notoriously sensitive to hyperparameters; runs can collapse, and diagnosing whether the fault lies in the policy, the reward model, or the KL coefficient is genuinely difficult.

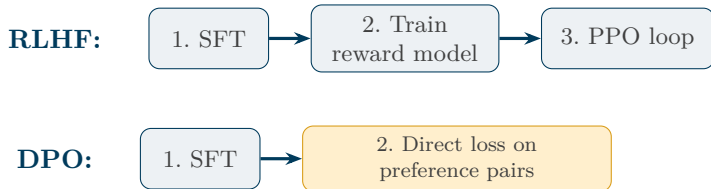
These pain points made the field ask: is the reinforcement-learning machinery actually necessary, or just the reward signal?

DPO (Rafailov et al., 2023) showed that, under the KL-constrained objective, **the language model is secretly its own reward model**: the optimal policy and the reward are algebraically interchangeable. That collapses the pipeline.

- No separate reward model. No reinforcement-learning loop. Preference pairs feed a *classification-style* loss, optimized by ordinary backpropagation:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

- Read it as: increase the likelihood margin of the preferred response over the rejected one, *relative to the reference model*, with β playing the same tethering role the KL term played in RLHF.
- Two models in memory instead of four, and standard supervised-training stability.



- **DPO:** simpler, cheaper, stable; now the default for open-weight preference tuning.
- **RLHF/PPO:** retains advantages when you need *online* exploration (training on fresh generations rather than a fixed preference set) or when the reward comes from something other than pairwise preferences, e.g. a verifier or unit tests, the setting the next frames develop.
- Both need the same scarce asset: high-quality, consistent **preference data**. Annotation quality usually decides the outcome more than the choice of objective.

GRPO (Shao et al., 2024) keeps the reinforcement-learning recipe but removes PPO's separate **value model**.

- For each prompt, sample a **group** of G responses (e.g., $G = 16$) from the current policy and score every one, giving rewards $r_1, \dots, r_G$.
- The advantage of response i is its reward standardized *within its own group*:

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

The group average plays the baseline role the value model used to fill, so a whole trained network disappears from the pipeline.

- Three models in memory instead of four, with the same clipped update and KL tether as PPO.
- GRPO trained the DeepSeek-R1 reasoning models and pairs naturally with the verifiable rewards on the next slide.

RLVR: Reinforcement Learning with Verifiable Rewards

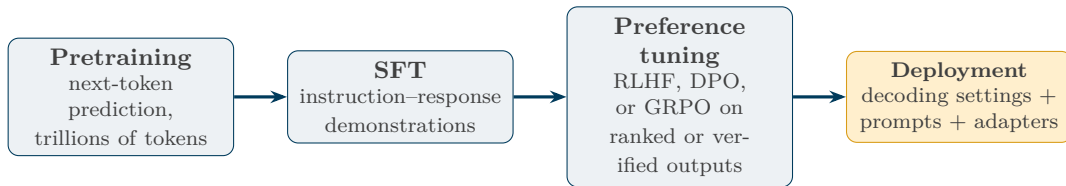
7 Alignment: Learning from Preferences

When correctness can be **checked by a program**, the learned reward model disappears too (Lambert et al., 2024).

- **Math**: compare the final answer to the known solution. **Code**: run the generated program against unit tests. **Logic**: hand the output to a formal verifier. The reward is binary and exact.
- Because the reward is computed by a checker instead of learned from annotations, there is no proxy to exploit: sycophancy and confident verbosity earn a plain 0.
- RLVR paired with GRPO produced the recent wave of **reasoning models**: reinforcement learning on verifiable problems teaches the model to generate long chains of thought, check them, and revise before answering.
- The boundary is sharp: RLVR needs a checkable answer. Essay quality, helpfulness, and tone still need a human or a model as the judge.

Human preference labels are the expensive ingredient, so **RLAIF** (Reinforcement Learning from AI Feedback) replaces the human annotator with a strong LLM (Bai et al., 2022; Lee et al., 2024).

- **LLM-as-a-judge**: a capable model receives the prompt, a rubric, and two candidate responses, and picks the better one. Its verdicts train the reward model, or feed DPO directly.
- Constitutional AI (Bai et al., 2022) is the canonical example: the judge critiques and ranks outputs against an explicit written list of principles.
- Preference data collection scales by orders of magnitude at a fraction of the cost, and on helpfulness benchmarks RLAIF lands close to human-feedback quality.
- Caveats: judge models favor verbose, confident answers, prefer their own outputs (self-preference bias), and pass their systematic errors on to every model trained against them.



- Every “assistant” model you use went through some version of this pipeline; model cards distinguish *base*, *instruct* (SFT), and *chat* (preference-tuned) checkpoints along exactly these lines.
- Everything in today’s lecture slots in somewhere: decoding and prompting at deployment time, SFT and PEFT in the middle, preference optimization at the end.
- PEFT composes with all of it: LoRA adapters can carry the SFT stage, the DPO stage, or both.

1. A language model outputs **distributions**; the decoding strategy (greedy, beam, temperature, top- k , nucleus) turns them into text.
2. **Temperature** controls sampling entropy via $p_i \propto \exp(\log p_i / T)$; **nucleus sampling** truncates the tail adaptively.
3. **Prompting is conditioning**: few-shot examples, chain-of-thought, and system prompts reshape $p(\text{output} \mid \text{prompt})$ with zero weight updates, but pay per-call cost.
4. **SFT** clones demonstrated behavior with cross-entropy loss masked to response tokens; data quality dominates data quantity.
5. **LoRA** learns a low-rank patch $W' = W + \frac{\alpha}{r}BA$, training $< 1\%$ of parameters; **QLoRA** runs it through a 4-bit NF4 base, putting a 7B fine-tune on a laptop.
6. **Quantization** trades bits for memory and speed at almost no quality cost, once outlier channels are handled (LLM.int8, GPTQ, AWQ).
7. **Preference optimization** (RLHF, DPO) trains on *rankings* rather than demonstrations; GRPO with verifiable rewards (RLVR) drives today's reasoning models, and LLM judges (RLAIF) scale the annotation.

Coming up. Weights are frozen at inference time and their knowledge has a training cutoff. Next we give the model access to *your* documents: retrieval-augmented generation, embeddings for search, and vector databases, the middle ground between prompting and fine-tuning that we skipped today.

- **Homework:** Homework 9 is due **July 24** on Blackboard.
- **Companion notebooks:** go through the AG News case-study notebook, then try to run the Unsloth counterpart on Colab (a free T4 runtime is enough).

Questions? waleakinfaderin@gwu.edu